

Unity ↔ Python WebSocket 通信ガイド

— アタッチ方法と JSON の使い方 —

1 概要

Unity を WebSocket クライアント、Python をサーバーとして、JSON やテキストを双方向にやり取りする。通信文字コードは UTF-8。

Python (サーバー)	ws_server モジュール + server_gui.py (Tkinter GUI)
Unity (クライアント)	UnityWebSocketClient.cs + keyboard_action.cs

- サーバーは通常 0.0.0.0:8765 で待ち受ける（全インターフェース待受の意味）。
- Unity 側の接続先 IP には、Python を動かす PC の**実際の LAN 内 IPv4**を設定する。
- 同一 PC で動かすなら Unity の接続先 IP は 127.0.0.1 でよい。
- 0.0.0.0 は「待受」専用の値。Unity の接続先には使わない。

2 ファイル一覧と保存場所

リポジトリ kokoro-shogi/ 直下からの相対パス。

ファイル	保存場所	役割
server.py	src/kokoro_shogi/server/	サーバー本体モジュール
server_gui.py	scripts/	操作用 GUI（これを起動する）
pyproject.toml	リポジトリ直下	Python 依存の管理
UnityWebSocketClient.cs	unity/.../Scripts/Net/	Unity 通信クライアント
keyboard_action.cs	unity/.../Scripts/Net/	キー操作の動作確認用
サンプル JSON	json/ 等	送信テスト用データ

Unity 側の完全パスは unity/KokoroShogi/Assets/_Project/Scripts/Net/。

3 Python 側の動かし方

3.1 準備と起動

```
cd kokoro-shogi
uv sync                # 依存を取得（初回のみ）
uv run python scripts/server_gui.py # GUI を起動
```

GUIで待受 IP 0.0.0.0 / ポート 8765 にして「サーバー起動」を押す。初回は Windows Firewall の許可（プライベートネットワーク, TCP 8765）が必要。

3.2 GUIでできること

- サーバーの起動／終了、待受 IP・ポートの指定。
- Unity が接続すべきローカル IP・URL の表示。
- 接続中クライアント IP の一覧表示。
- 選択したクライアントへ「テキスト」または「JSON ファイル」を送信（「参照」でファイル選択）。
- Unity から届いた `{"text": "..."}` の本文を「受信テキスト」欄に表示。それ以外の JSON は「受信・接続ログ」に表示。

起動すると GUI 上部に接続先が表示される:

接続先:	<code>ws://192.168.1.20:8765</code>	← 別 PC の Unity にはこれ
同一 PC から:	<code>ws://127.0.0.1:8765</code>	← 同じ PC の Unity にはこれ

3.3 GUIを使わずコードから動かす

サーバーはモジュールとして直接呼べる（第 6.3 節の Python 例を参照）。

4 Unity ファイルの配置

2つの C#ファイルは既にリポジトリの下記に含まれる。自分で追加する場合もファイル名とクラス名を必ず一致させる（MonoBehaviour の必須条件。ダウンロードで ... (1).cs になっていたらリネーム）。

unity/KokoroShogi/Assets/_Project/Scripts/Net/	
└── UnityWebSocketClient.cs	→ class UnityWebSocketClient
└── keyboard_action.cs	→ class keyboard_action

Unity 側の前提: ClientWebSocket / 新 Input System / DontDestroyOnLoad を使用。新 Input System は Package Manager で導入し、Project Settings > Player > Active Input Handling を Input System Package (New) か Both にする。

5 アタッチ方法 (Unity)

5.1 UnityWebSocketClient.cs

1. 最初のシーンで Hierarchy を右クリック → Create Empty。名前を WebSocketManager 等にする。
2. UnityWebSocketClient.cs をその GameObject へドラッグしてアタッチ。
3. Hierarchy のルートに置く（DontDestroyOnLoad を使うため、他 Object の子にしない）。
4. Inspector で下表を設定する。

項目	内容
Server Ip	Python サーバー PC の LAN 内 IPv4 (同一 PC なら 127.0.0.1)
Server Port	8765
AutoConnect	ON で登場時に自動接続
EchoSignal	ON で受信後に <code>success_echo</code> を返信
Echo Preview Length	echo に含める受信内容の先頭文字数
On Message Received	受信時のコールバック登録
On Connection Status Changed	接続状態変更時のコールバック登録

シングルトンで参照でき、Button の `On Click()` からでも呼べる。

```
UnityWebSocketClient.Instance.Connect();    // 接続
UnityWebSocketClient.Instance.Disconnect(); // 切断
```

AutoConnect が OFF の場合は `Start()` で自動接続せず、上記 `Connect()` が呼ばれたときに接続する。

5.2 keyboard_action.cs (動作確認用)

1. 空の GameObject (例 KeyboardAction) を作りアタッチ。
2. Inspector の Web Socket Client 欄に WebSocketManager をドラッグ (未設定なら実行時に `UnityWebSocketClient.Instance` を自動取得するが、明示設定を推奨)。

キー操作:

キー	動作
C	現在の IP/Port へ接続
D	切断
S	テストメッセージ送信 (押すたびに回数が増える)
I / P	IP / Port 変更モード開始
Enter / Esc	入力確定 / キャンセル

接続中に IP/Port を変えても現在の接続は切り替わらない。D → C で再接続する。

6 JSON の使い方

6.1 基本ルール

- テキストを送ると、両側とも `{"text": "..."}` 形式に自動変換される。
- JSON オブジェクト／配列の文字列を渡すと、**そのまま**送信される。
- Python は、JSON ファイル送信時に一度パースして妥当性を確認してから送る。
- 既定の最大受信サイズは 64 MiB。

6.2 Unity → Python 送信

```
// テキスト送信 → 実際は {"text":"success"} として届く
UnityWebSocketClient.Instance.SendText("success");

// JSON 文字列をそのまま送信
UnityWebSocketClient.Instance.SendRawJson(
    "{\"type\":\"result\",\"success\":true}"
);

// UI 入力欄から接続先を設定して接続
UnityWebSocketClient.Instance.ConfigureAndConnect(
    ipInput.text, portInput.text
);
```

6.3 Python → Unity 送信

GUIから「テキスト」または「JSON ファイル」を選び、接続中クライアント IP を指定して送信する。Python モジュールを直接使う例:

```
import time
from kokoro_shogi.server import server as ws_server

ws_server.start_server() # 0.0.0.0:8765

try:
    while True:
        clients = ws_server.get_connected_clients()
        if clients:
            ip = clients[0]
            ws_server.send_client(ip, "success") # {"text":"success"}
            ws_server.send_client(ip, "sample.json") # ファイル内容を送信
            got = ws_server.receive_client(ip)
            if got is not None:
                print("Unity から受信:", got)
            time.sleep(0.1)
finally:
    ws_server.end_server()
```

主な公開関数:

関数	説明
<code>start_server(host, port)</code>	サーバー起動（起動済なら False）
<code>end_server()</code>	全接続を閉じて終了
<code>send_client(ip, data)</code>	指定 IP へ送信（テキスト/JSON ファイルパス）
<code>receive_client(ip)</code>	指定 IP から受信（無ければ None）
<code>receive_any()</code>	送信元を問わず受信（client_ip,data）
<code>get_connected_clients()</code>	接続中 IP 一覧

※ 接続一覧は IP 単位。同一 IP から複数接続があると `send_client()` はその全接続へ送る。

6.4 データ形式の例

テキスト（自動変換後の最小形）：

```
{ "text": "メッセージ内容" }
```

本プロジェクトの実際のメッセージは Schema 1.0。全て `schema` と `type` を持つ JSON オブジェクトで、`type` で種類を判別する。

type	方向	用途
game_control	Unity → AI	対局の開始/操作
state_update	AI → Unity	局面・駒の状態（毎手）
legal_moves	AI → Unity	合法手リスト（ハイライト用）
move_request	Unity → AI	人間が選んだ手
career	AI → Unity	対局終了時の通算成績

対局の流れ:

1. Unity → AI: `game_control("start")` で対局開始。
2. AI → Unity: `state_update`（初期局面）と `legal_moves`。Unity は指せる手だけハイライト。
3. 人間が駒をタップ → Unity → AI: `move_request`。
4. AI が思考して指し、AI → Unity: 新しい `state_update`（人間の手・AI の手で計 2 回）。
5. 2～4 を終局まで繰り返し、終局時に AI → Unity: `career`。

game_control (Unity → AI) :

```
{ "schema": "1.0", "type": "game_control", "command": "start" }
```

move_request (Unity → AI) :

```
{
  "schema": "1.0",
  "type": "move_request",
  "move": { "from": "77", "to": "76", "promote": false }
}
```

legal_moves (AI → Unity)。マス目は筋段 2 桁、持ち駒からの打ちは `from:"00" + drop_species`:

```
{
  "schema": "1.0",
  "type": "legal_moves",
  "moves": [
    { "from": "77", "to": "76", "promote": false },
    { "from": "88", "to": "22", "promote": true },
    { "from": "00", "to": "55", "drop_species": "FU" }
  ]
}
```

state_update (AI → Unity)。局面は `sfen`、各駒に `mood/desire` 等の内部状態が付く（`pieces` は一部抜粋）：

```
{
  "schema": "1.0",
  "type": "state_update",
  "ply": 42,
  "sfen": "ln2g1snb/r1k2g2l/... b - 43",
  "last_move": {
    "from": "32", "to": "23",
    "piece_id": "G49_gen0_0018",
    "capture": false, "promote": false, "drop": false
  },
  "eval": -0.38,
  "pieces": [
    {
      "piece_id": "B22_gen0_0006",
      "species": "KA", "owner": 1, "square": "11",
      "mood": { "fear": 0.175, "aggression": 0.15, "valence": 0.34 },
      "desire": { "survive": 0.387, "attack": 0.15, "promote": 0.0,
                  "defend": 0.0, "advance": 0.0, "redeploy": 0.0 },
      "alpha": 0.051,
      "relations": [ { "to": "P33_gen0_0012", "r": 0.6 } ]
    }
  ]
}
```

career (AI → Unity, 終局時) :

```
{
  "schema": "1.0",
  "type": "career",
  "pieces": [
    { "piece_id": "R28_gen0_0011", "species": "HI",
      "games": 120, "survival_rate": 0.78,
      "promotions": 55, "mvp_count": 21 }
  ],
  "last_game_mvp": { "piece_id": "R28_gen0_0011", "contribution": 2.3 }
}
```

6.5 受信と Echo

Unityは受信すると Console に [WebSocket RECEIVE] {...} を表示し、On Message Received に受信文字列を渡す。EchoSignal が ON なら、受信後に success_echo[先頭部分...] を返信する（実際の送信は {"text": "success_echo[...]"}）。Python GUI では {"text": "..."} は「受信テキスト」欄に、それ以外の JSON は「受信・接続ログ」に表示される。

7 最短動作確認

1. Python: server_gui.py を起動 → 待受 0.0.0.0:8765 で「起動」。表示されたローカル IP を控える。
2. Unity: WebSocketManager/KeyboardAction をアタッチし、Server Ip に上記 IP、Server Port に 8765。

3. Play → (AutoConnect OFF なら) C で接続。GUI のクライアント一覧に Unity の IP が出るか確認。
4. Unity で S → GUI に「テストメッセージ 1 回目」が出るか確認。
5. GUI からテキスト/JSON を送信 → Unity Console に [WebSocket RECEIVE]。
6. Echo が ON なら GUI に `success_echo[...]` が返る。

8 ひとりで (1 台の PC で) 試す

別 PC を用意せず、1 台で送受信を確認する方法。ポイントは Unity の接続先 IP に 127.0.0.1 (同じ PC の自分自身) を使うこと。

8.1 方法 A : Unity + Python を同一 PC で動かす (推奨)

1. Python: `uv run python scripts/server_gui.py` を起動し、待受 0.0.0.0:8765 で「サーバー起動」。
2. Unity: `UnityWebSocketClient` の Server Ip を 127.0.0.1、Server Port を 8765 にする。
3. Unity を Play し、AutoConnect が OFF なら C キーで接続。GUI のクライアント一覧に 127.0.0.1 が出れば接続成功。
4. **Unity → Python:** Unity で S キー → GUI の受信テキストに「テストメッセージ 1 回目」が出る。
5. **Python → Unity:** GUI で 127.0.0.1 を選び、テキストや `json_/2_legal_moves.json` 等を「参照」して送信 → Unity Console に [WebSocket RECEIVE] が出る。

これで Unity・Python 双方向をひとりで確認できる。

8.2 方法 B : Unity を使わず Python だけで送信を確認

Unity がまだ無い段階でも、GUI を起動してサンプル JSON を「参照」→送信する操作までは確認できる。ただし受信側 (クライアント) が接続していないと実際には送られない。受信相手が要するため、送受信の往復を試すには方法 A が必要。サーバー起動や GUI 操作・ファイル選択の動作確認だけなら Unity 無しでも可。

8.3 別 PC から試す場合 (参考)

同一 LAN 内の別 PC・スマホ実機で試すときは、127.0.0.1 ではなく GUI に表示されたローカル IP (例 192.168.1.20) を Unity に設定する。両 PC が同じ LAN にあり、Firewall で 8765 が許可されている必要がある。

9 トラブルシューティング

- **接続できない:** サーバー起動済か／Unity に IP (0.0.0.0 でなく実 IP) と Port が正しいか／同一 LAN か／Firewall 許可／VPN が別 IP を優先していないか。
- **Input System エラー:** 旧 API (`Input.GetKeyDown` 等) を使う別スクリプトの残存を確認。本スクリプトは新 Input System。
- **echo が返らない:** EchoSignal ON、[WebSocket RECEIVE] / SEND SUCCESS の表示、受信直後に切断していないかを確認。

注意: 本実装は開発・検証用で LAN 内前提。通信は非暗号の `ws://`。インターネット越しに使う場合は認証・入力検証・アクセス制限・`wss://`(TLS) を追加すること。また本 Unity 実装は `ClientWebSocket` 利用可能な Editor/Standalone 向け。WebGL では別実装 (JS 側 WebSocket) が必要。